

Selecting Correct Programs under a Noisy Verifier

Exploiting Shared Structure for Consistency and Correctness

Problem

We synthesise, for each natural-language question q , a program p_q that maps a trajectory x to an answer in $\{\text{YES}, \text{NO}, \text{MAYBE}\}$. We have *no* solid way to verify that p_q correctly implements q . One available verifier is an LLM call

$$V_{\text{llm}}(q, p_q, \{(x_i, p_q(x_i))\}_i) \rightarrow \{0, 1\},$$

which judges correctness from the question, the program, and input/output pairs. It is *noisy*. Under V_{llm} there is a class $P = \{p_q\}$ of plausible (high-probability) programs, and we must select the correct ones from within it.

The extra leverage: questions and their programs refer to *shared objects* — “the white cross,” “the highlighted 3×3 grid,” “a perimeter square.” Independently synthesised programs each re-derive a checker for such an object; we instead want a checker that is *consistent across the whole universe of questions and programs*. The goal is to write down objectives that turn that shared structure into a correctness signal.

Core reframe

Stop verifying each p_q in isolation. Treat correctness as a **joint inference problem over a population of programs that share latent concepts**, where agreement on the shared latents is a label-free signal. V_{llm} becomes a noisy *prior* per program, not the decision procedure; the decision comes from coupling programs together. Four sources of signal:

- $V_{\text{llm}}(q, p_q)$ — noisy local plausibility;
- **cross-program agreement** on shared sub-concepts (the structural signal);
- **logical coherence** of the *answers* under a single world model;
- **behavioral grounding** — a subset of concepts have observable consequences in the trajectory (a partial oracle).

1 Factored synthesis over a shared concept library (MDL)

Define a library $L = \{c_1, \dots, c_m\}$ of concept sub-programs — `highlighted_grid`, `white_cross`, `perimeter_cells`. Require each question-program to be a composition

$$p_q = g_q \circ (c_{i_1}, \dots, c_{i_k}),$$

with question-specific glue g_q over library calls. MDL / Bayesian objective:

$$\max_{L, \{p_q\}} \sum_q \log V_{\text{llm}}(q, p_q) - \lambda |L|.$$

The $-\lambda|L|$ (description-length) term *forces* sharing: a concept used by five programs is paid for once, so there is pressure toward a single realisation (this is DreamCoder’s wake/compression loop). The point is not compactness — it is that **a shared c has its posterior sharpened by the union of all questions that touch it**, strictly more evidence than per-program V_{llm} . MDL alone is syntactic, though; it does not know whether the shared concept is *correct*. Hence the next term.

2 Cross-program agreement on shared latents (the key signal)

This is co-training / multi-view consistency, and it needs no oracle. For a shared concept c , extract the *intermediate value* each program computes for it on every real state x_i . If two programs both reference “the highlighted 3×3 grid,” each emits an index $c_j(x_i)$, $c_k(x_i)$. Define

$$\mathcal{L}_{\text{consist}}(c) = \sum_i \sum_{j,k \text{ use } c} \mathbb{I}[c_j(x_i) \neq c_k(x_i)]$$

Minimising total disagreement over the *choice of which candidate implementation each program uses* selects implementations that agree about the shared object on the actual data. The concept is the shared latent; each question is a different “view”; agreement across views is self-supervised.

Why this fixes the prototype failure. Program Q01 invented “perimeter = the 64×64 outer edge.” Any other program referencing “perimeter square” that encoded it as the border of a 3×3 grid would produce a *different mask on the same states* — the disagreement flags the wrong encoding with no ground truth. Where a hand-written probe was a privileged adjudicator, here **every other program that touches the concept is a probe**.

3 Answer-level coherence under a single world model

Questions are logically entangled: “can red revert to blue?” and “can blue become anything but red?” both constrain the same transition relation. A correct set of answers must be jointly satisfiable by one transition model M :

$$\text{Coherence}(\{a_q\}) = \max_M \sum_q \mathbb{I}[a_q \text{ consistent with } M].$$

In practice no full SAT is needed: extract pairwise entailment/exclusion edges between questions once (an LLM produces the relation graph cheaply), and penalise answer assignments that violate them. This is the only term that looks *across* questions at the answer level, so it catches contradictions per-program verification structurally cannot see.

4 Truth discovery to fold in the noisy verifier

To *use* V_{llm} and the legacy log-reading answers without trusting any of them, run Dawid–Skene / truth-discovery EM. Treat each source (log-reader, each candidate program, the verifier) as a noisy estimator of a latent truth a_q^* with its own reliability θ_s (a confusion matrix). EM jointly estimates $\{a_q^*\}$ and $\{\theta_s\}$: sources that track consensus gain weight, and the consensus shifts toward reliable sources. This requires *independent* estimators — so synthesise k candidate programs per question at varied temperature/prompt, which Section 2 wants anyway.

5 Unified objective

$$\max_{L, \{p_q\}} \underbrace{\sum_q \log V_{\text{llm}}(q, p_q)}_{\text{local prior}} + \beta \underbrace{\sum_c (-\mathcal{L}_{\text{consist}}(c))}_{\text{structural agreement}} + \gamma \underbrace{\text{Coherence}(\{a_q\})}_{\text{answer logic}} - \lambda \underbrace{|L|}_{\text{reuse pressure}}$$

Each term covers a blind spot of the others: V_{llm} keeps programs on-topic, MDL creates the sharing, agreement validates the shared concepts on data, coherence validates the answers against each other.

Optimisation is block-coordinate ascent / EM over a discrete space:

1. *Propose*: sample k candidate programs per question from the LLM (diversity matters).
2. *Select & refactor (M-step)*: find sub-programs that, if unified into one library entry, reduce both $|L|$ and $\mathcal{L}_{\text{consist}}$ — DreamCoder compression, but the refactor is *validated by agreement on real trajectory data*, not just syntactic anti-unification.
3. *Re-estimate (E-step)*: given the library, reselect each p_q ’s implementation and re-run truth discovery on the votes.

6 Identifiability caveat — and why this environment escapes it

Agreement is necessary, not sufficient: programs can *agree on a wrong concept* (all encode perimeter wrong, identically) — standard co-training collusion. Three anchors break it; the last is specific to this setting.

1. **Semi-supervised anchoring** — pin a few concepts to hand-written structural probes / a validated perception layer. Agreement then *propagates* grounding from anchored to unanchored concepts.
2. **Generation diversity** — independent derivations colluding on the same wrong mask is unlikely under varied prompts; collusion probability is what one is implicitly bounding.
3. **Behavioral grounding (the partial oracle we already have)**. Some concepts have observable consequences. “Highlighted grid” should be *the grid where clicks change cells*; clicking elsewhere should not. So a concept implementation can be scored against the actual transitions: `highlighted_grid(x)` is correct iff the cells that change under the logged click fall inside it. This grounds a subset of concepts directly, and agreement carries it to the rest. It is the deepest signal — the environment, not the LLM, adjudicates — and it is what makes the scheme more than mutual hallucination.

Throughline. The earlier prototype concluded “make predicates call a validated perception layer.” These formulations say the validation signal for that layer is **cross-question agreement plus behavioral grounding**, so the layer can be *learned* rather than hand-labelled — which is what is needed to scale past a handful of questions.